

Ricerca sulle tecniche di offuscamento dell'Hardware

Dimitri Binosi¹

¹Università degli Studi di Verona

25 agosto 2026

Sommario

L'offuscamento dell'hardware è una tecnica impiegata in ambito tecnologico per mantenere segreto il funzionamento di un circuito integrato ed IP in esso contenuto. Nonostante l'esistenza di tali metodologie la ricerca continua ad individuare nuove tecniche di attacco, rendendo quindi necessario un costante aggiornamento delle tecniche di protezione. In questo paper, viene riportato lo stato dell'arte dei principali attacchi e delle tecniche di difesa più efficaci.

1 Introduzione

Nel panorama odierno, il mercato delle componenti elettroniche contraffatte è responsabile di ingenti perdite economiche da parte delle aziende proprietarie. L'origine di queste componenti è variegata, alcune provengono da sovrapproduzioni alla fonderia con cui la casa madre ha il contratto, altre sono piratate, altre ancora prodotte da terze parti seguendo progetti rubati. Nel contesto automobilistico, il problema assume una rilevanza maggiore poiché una vulnerabilità nel circuito di controllo potrebbe tradursi in danni a persone. Come contromisura, diverse aziende hanno implementato tecniche di offuscamento dell'hardware, le quali, ne permettono il corretto utilizzo solo se autorizzato, preservando in questo modo la privacy di tale tecnologia.

In questa ricerca viene trattato il sopracitato problema, con particolare riferimento ai circuiti digitali integrati ed alle metodologie di offuscamento. Nel primo capitolo (1) viene introdotta una semplice visione del problema e del motivo per cui la privacy di tale tecnologia è necessaria. Nel capitolo 2 si spiega la natura della soluzione proposta, l'offuscamento

dell'hardware. Nella sezione 3, vengono esposti degli attacchi utilizzati per oltrepassare l'offuscamento, soffermandoci sul più potente tra questi. Infine, nella sezione 4, viene presentato lo stato dell'arte delle contromisure sviluppate.

2 Offuscamento dell'Hardware

2.1 Descrizione generale

L'offuscamento dell'hardware mira a rendere difficile la violazione della tecnologia a cui viene applicato; a seconda della tipologia del dispositivo interessato, viene suddiviso in due categorie [1]:

- *Offuscamento del core DSP*: questo tipo di offuscamento implementa un grado eccezionalmente elevato di trasformazione del grafo dei flussi di dati (DFG) rappresentativo del core DSP (Digital Signal Processing), per convertirlo in un formato oscuro che riflette una struttura poco chiara a livello RTL (Register Transfer Level) o a livello di porte. È anche noto come *offuscamento strutturale*.

Una variante di questa tecnica è l'*offuscamento funzionale*, che prevede l'inserimento di bit di chiave. Un insieme di blocchi di locking AES e IP (ILB) impedisce quindi al core DSP di funzionare correttamente se non viene usata la sequenza di chiavi corretta.

- *Offuscamento combinatorio*: si tratta di una modifica del circuito a livello strutturale (introduzione di porte logiche come MUX o XOR) servendosi poi di una chiave

segreta per l'esecuzione corretta del circuito (quest'ultima estratta da una memoria sul chip a prova di attacco).

È inoltre possibile distinguere l'offuscamento dell'hardware in *attivo* e *passivo*: il primo, a differenza del secondo, comporta un'alterazione delle funzionalità del sistema. L'approccio attivo, spesso è basato su chiavi e può essere applicato a diversi livelli, in questo paper ci focalizziamo su RTL (Register Transfer Level).

Per raggiungere questo tipo di offuscamento, dobbiamo prima dare un significato al termine logic locking [2]: la capacità di aggiungere una programmabilità dopo la fabbricazione, che può essere introdotta usando alcuni gate aggiuntivi, noti come key-programmable gates (key gates), controllati dal segreto del logic locking, cioè la chiave. L'aggiunta della programmabilità dopo la fabbricazione spesso viene inserita tra la sintesi logica del circuito e quella fisica, a livello di porte logiche (opzione preponderante, più del 90% dei casi).

Esistono tre diversi tipi di logic locking, rispettivamente basati sulla porta logica MUX, quella XOR oppure sulla look-up-table (LUT).

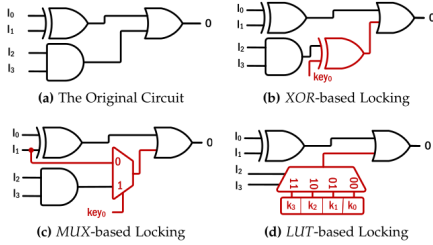


Figura 1: Logic Locking

2.2 Tipologia attacco

Si assume che l'attaccante disponga di un circuito sbloccato, sul quale sia possibile applicare diversi vettori di input e osservarne le corrispondenti uscite, oltre alla maschera del circuito stesso.

Nel seguito, ci si concentrerà su attacchi rivolti a circuiti protetti mediante offuscamento combinatorio. Data la natura basata su chiave di tale approccio, un primo tentativo di recupero della *secret key* potrebbe consistere in un attacco *brute force*; tuttavia, la complessità di tale strategia è pari a $O(2^N)$, con N il numero di bit della chiave.

Un'alternativa consiste nell'impiego di strumenti di ATPG (*Automatic Test Pattern*

Generation), che permettono di sfruttare specifici pattern di input per inferire i bit della chiave. In assenza di pattern significativi, tuttavia, tale approccio risulta inefficace.

Un ulteriore esempio è rappresentato dagli attacchi basati su *SAT solver*, i quali, a partire da un insieme di input, sono in grado di determinare una chiave che li soddisfi simultaneamente. Per garantirne la correttezza, però, sarebbe necessario verificare tutti i possibili input, ossia 2^M combinazioni, rendendo l'approccio computazionalmente impraticabile.

Nel prossimo capitolo verranno quindi presentate tecniche che, sfruttando principi relativamente semplici, consentono di violare l'offuscamento e recuperare la chiave segreta.

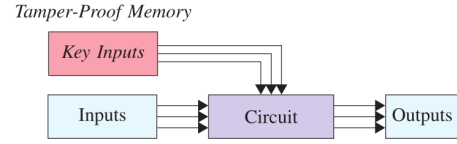


Figura 2: Logic Encryption

3 Attacchi all'offuscamento

3.1 Attacco SAT

Uno degli attacchi più efficaci proposti negli ultimi anni, è basato su SAT-solver, da cui prende anche il nome, proposto in [3]. L'algoritmo di attacco è di natura ibrida e sfrutta, un approccio SAT e pochi, ma precisi, input per verificare il risultato associato nel circuito sbloccato, al fine di indovinare la key e sbloccare il circuito.

L'algoritmo 1, utilizza F per memorizzare i requisiti della chiave mano a mano che vengono scoperti, come indicato al passo 6. Il ciclo *while*, ci assicura la permanenza nel loop finché sarà presente un input capace di causare un diverso output (sul circuito bloccato) su almeno una coppia di chiavi. L'obiettivo è infatti restringere progressivamente lo spazio delle chiavi alla sola classe di equivalenza delle chiavi corrette, escludendo tutte le altre. Questo procedimento risulta più efficiente rispetto a un attacco brute force, poiché a ogni iterazione viene eliminata un'intera classe di equivalenza di chiavi, ossia tutte quelle chiavi che, a fronte di un input fissato, non producono un determinato output (riga 6).

Algorithm 1 Algoritmo

Require: Circuito bloccato C , Circuito sbloccato $eval$

Ensure: Key K_C

```
1:  $i \leftarrow 1$ 
2:  $F_1 \leftarrow C(X_d, K_1, Y_1) \wedge C(X_d, K_2, Y_2)$ 
3: while  $\text{sat}(F_i \wedge (Y_1 \neq Y_2))$  do
4:    $X_d \leftarrow \text{sat\_assignment}(F_i \wedge (Y_1 \neq Y_2))$ 
5:    $Y_d \leftarrow eval(X_d)$ 
6:    $F_{i+1} \leftarrow F_i \wedge C(X_d, K_1, Y_d) \wedge C(X_d, K_2, Y_d)$ 
7:    $i \leftarrow i + 1$ 
8: end while
9:  $K_C \leftarrow \text{sat\_assignment}_{K_1}(F_i)$ 
10: return  $K_C$ 
```

Nel paper originale, vengono riportati alcuni benchmark eseguiti su circuiti criptati con algoritmi ormai deprecati, tuttavia, in uno di questi esperimenti si osserva una potenziale vulnerabilità di SAT-attack, un *AND-tree*, ossia una struttura ad albero composta di porte logiche AND. Si supponga che il circuito offuscato sia:

$$y_e = \bigwedge_{i=1}^N (x_i \oplus k_i)$$

Si osserva che ogni assegnamento di $k_1, \dots, k_N$ è una classe di equivalenza. Ne consegue che un *AND-tree* con N inputs, criptato come sopra, presenta 2^N classi di equivalenza per le chiavi. Nonostante ciò, il circuito è stato comunque decifrato, evidenziando i limiti pratici di tale vulnerabilità.

3.2 Attacchi alternativi

In questa sottosezione introduciamo brevemente un altro attacco e un tool che verranno ripresi nella prossima sezione: il removal attack [4] e Valkyrie [5], un tool di valutazione della vulnerabilità per tecniche di logic locking. Il removal attack si basa sull'idea di individuare segnali o unità critiche nella struttura del circuito locked e di rimuoverli per recuperare il comportamento originale del design; in particolare, può permettere di ricostruire il circuito originale eliminando il locking unit in alcune famiglie di logic locking. Valkyrie, invece, è un security diagnostic tool che analizza netlist originali e locked per identificare vulnerabilità strutturali e segnali critici suscettibili a circuit-recovery attacks.

4 Difese contro gli attacchi

Con l'avvento del SAT-attack sono sorte nuove metodologie di contrasto, divise in due categorie, a seconda dell'approccio: da un lato

quelle che sfruttano la natura dei SAT-solvers per prolungare il tempo di computazione per iterazione, e dall'altro approcci che inseriscono una point-function nel circuito, aumentando significativamente le iterazioni necessarie per deoffuscare l'obiettivo.

La prima soluzione crea blocchi SAT-hard, resistenti agli attacchi SAT, ma caratterizzati da un notevole overhead in superficie del circuito e deboli contro il removal-attack. La seconda categoria, si basa invece su point-function, funzioni booleane che assumono valore 1 per un solo specifico input. Quest'ultima, si divide ulteriormente in funzioni: in-cone, se già presenti nel circuito originale, e out-of-cone, se aggiunte durante il processo di cifratura.

È inoltre opportuno sottolineare l'esistenza del remove-all DIP (Distinguish Input Pattern), un pattern di input che può consentire all'attaccante SAT di individuare rapidamente la chiave segreta.

Al fine di affrontare simultaneamente, l'attacco SAT, Valkyrie, remove-attack e il problema dei DIP, è stata proposta la tecnica di difesa two-level-based [6]. Quest'ultima, si basa su split-compensate, un'operazione capace di espandere l'*AND-tree* introducendo variabili esterne ed interne. Le variabili esterne aumentano la dimensione della struttura senza modificare il numero di classi di equivalenza delle chiavi errate, mentre quelle interne mirano a ridurre il numero di remove-all DIP.

Nella figura 3 viene presentato in forma schematica il flusso di esecuzione: si introducono variabili interne o esterne a seconda del vincolo violato, cioè il threshold sul numero massimo di DIP oppure il limite sul numero di variabili.

Infine come è possibile osservare dalla figura 4 la tecnica proposta in [6], localizza un *AND-tree*, controlla se vengono violati i limiti prestabiliti e nel caso, applica l'operazione descritta precedentemente. Quando vengono rispettate entrambe le limitazioni si procede a criptare l'*AND-tree* ed all'ottimizzazione del cono logico. Questo metodo risulta efficace perché bilancia resistenza agli attacchi SAT e contenimento dell'overhead strutturale.

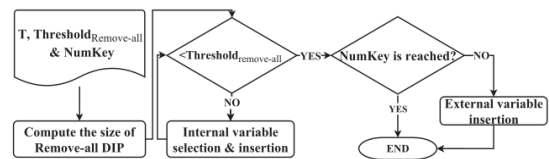


Figura 3: Flusso dell'operazione compensated-split

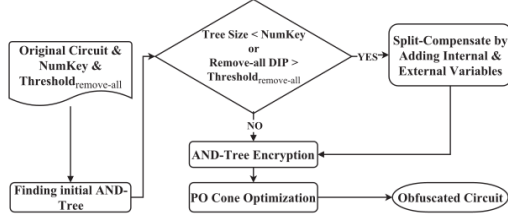


Figura 4: Flusso della tecnica two-level-based

Conclusione

Sebbene esistano altri contributi importanti non menzionati in questa ricerca, tramite pochi ma mirati esempi è stato possibile fornire un'idea generale dello stato dell'arte dell'offuscamento dell'hardware. Con la produzione di nuovi articoli ogni anno, il campo continuerà ad evolversi ed avanzare.

Riferimenti bibliografici

- [1] Khaled Saleh, Omar Darwish, and Fathi Amsaad. Hardware based obfuscation for secure and trusted systems: A comprehensive study. In *1st International Conference on Secure IoT*, 2025.
- [2] Hadi Mardani Kamali, Kimia Zamiri Azar, Farimah Farahmandi, and Mark Tehrani-poor. Advances in logic locking: Past, present, and prospects. *Future Hardware Security Research Series*, 2022.
- [3] Pramod Subramanyan, Sayak Ray, and Sharad Malik. Evaluating the security of logic encryption algorithms. In *2015 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, 2015.
- [4] Isaac McDaniel, Michael Zuzak, and Ankur Srivastava. Removal of sat-hard instances in logic obfuscation through inference of functionality. In *ACM Transactions on Design Automation of Electronic Systems*, 2024.
- [5] Nimisha Limaye, Satwik Patnaik, and Ozgur Sinanoglu. Valkyrie: Vulnerability assessment tool and attack for provably-secure logic locking techniques. In *IEEE Transactions on Information Forensics and Security*, 2022.
- [6] RuiJie Wang, Li-Nung Hsu, Yung-Chih Chen, and TingTing Hwang. Expanding and obfuscating in-cone trees to resist sat attack in logic locking. *IEEE Transactions on Reliability*, 2025.